

Delta Lake Incremental Processing Assignment

Objective: Perform incremental data processing using Delta Lake. The screenshots show dataset loading, cleaning, incremental data creation, MERGE operation, result validation, final output, and SCD Type 2 implementation.

Assignment steps covered:

1. Load dataset into a Delta table.
2. Perform basic cleaning: handle nulls and remove duplicates.
3. Create a second dataset simulating new/incremental data.
4. Apply MERGE operation to update existing and insert new records.
5. Validate results using row count, duplicate checks and null checks.
6. Display final dataset and summary.
7. Implement SCD Type 2 to maintain historical records.

Figure Index

Figure	Assignment Step	Figure Name
Figure 1	Step 1: Load dataset into a Delta table	Raw Superstore Delta table loaded and displayed
Figure 2	Step 2: Basic cleaning	Column name standardization for raw data
Figure 3.1	Step 2: Basic cleaning	Data cleaning code - remove duplicates, trim text columns and handle data types
Figure 3.2	Step 2: Basic cleaning	Cleaned data output and row-count validation
Figure 4	Step 1/2: Create cleaned Delta table	Cleaned DataFrame saved as a Delta table
Figure 5.1	Step 3: Create incremental data	Incremental dataset preparation - existing updates and new records
Figure 5.2	Step 3: Create incremental data	Incremental dataset output with updated and new records
Figure 6	Step 4: Apply MERGE operation	Delta Lake MERGE operation for SCD Type 1 upsert
Figure 7.1	Step 5: Validate results	Final table validation code - row count, duplicate check and null check
Figure 7.2	Step 5: Validate results	Validation output showing final row count and zero duplicate row IDs
Figure 8	Step 6: Display final dataset	Final SCD Type 1 Delta table after MERGE
Figure 9.1	Step 6: Display summary	Summary DataFrame creation for assignment metrics
Figure 9.2	Step 6: Display summary	Summary output for SCD Type 1 processing
Figure 10.1	Additional SCD Type 2 implementation	SCD Type 2 customer table creation from master data
Figure 10.2	Additional SCD Type 2 implementation	Initial SCD Type 2 table output and row count
Figure 11.1	Additional SCD Type 2 implementation	SCD Type 2 incremental data preparation
Figure 11.2	Additional SCD Type 2 implementation	SCD Type 2 incremental records displayed
Figure 12.1	Additional SCD Type 2 implementation	Hash-based change detection logic for SCD Type 2
Figure 12.2	Additional SCD Type 2 implementation	Records selected for insertion in SCD Type 2
Figure 13	Additional SCD Type 2 implementation	SCD Type 2 MERGE operation to expire old current records
Figure 14	Additional SCD Type 2 implementation	New SCD Type 2 version records inserted successfully
Figure 15.1	Additional SCD Type 2 implementation	Final SCD Type 2 table display after updates and inserts
Figure 15.2	Additional SCD Type 2 implementation	SCD Type 2 final row count and active/inactive record summary
Figure 16.1	Additional SCD Type 2 validation	Current-record duplicate validation for SCD Type 2
Figure 16.2	Additional SCD Type 2 validation	SCD Type 2 duplicate check output showing customers with more than one current record
Figure 17	Additional SCD Type 2 validation	Historical SCD Type 2 records with version greater than 1
Figure 18.1	Final assignment summary	Final assignment summary DataFrame creation

Step 1: Load dataset into a Delta table

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Spaces

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

delta_scd_assignment

File Edit View Run Help Python Last edit was 3 minutes ago

3 minutes ago

1

Python

```
df_raw = spark.table("workspace.default.sample_superstore")

display(df_raw)
print("Total rows:", df_raw.count())
print("Total columns:", len(df_raw.columns))
```

See performance (2) Checked for improvements

df_raw: pyspark.sql.connect.dataframe.DataFrame = [Row ID: long, Order ID: string ... 19 more fields]

Table

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment
1	1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer
2	2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer
3	3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate
4	4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer
5	5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer
6	6	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
7	7	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
8	8	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
9	9	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
10	10	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer

Figure 1: Raw Superstore Delta table loaded and displayed

Step 2: Basic cleaning

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Spaces

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

delta_scd_assignment

File Edit View Run Help Python Last edit was now

2 minutes ago (3s)

2

Python

```
import re

def clean_column_name(name):
    name = name.strip().lower()
    name = re.sub(r"[^a-zA-Z0-9]+", "_", name)
    name = re.sub(r"_+", "_", name)
    return name.strip("_")

df = df_raw

for old_col in df_raw.columns:
    df = df.withColumnRenamed(old_col, clean_column_name(old_col))

display(df)
print(df.columns)
```

See performance (1) ✓ Checked for improvements

df: pyspark.sql.connect.dataframe.DataFrame = [row_id: long, order_id: string ... 19 more fields]

Table

+

row_id

order_id

order_date

ship_date

ship_mode

customer_id

customer_name

segment

1

1

CA-2016-152156

2016-11-08

2016-11-11

Second Class

CG-12520

Claire Gute

Consumer

2

2

CA-2016-152156

2016-11-08

2016-11-11

Second Class

CG-12520

Claire Gute

Consumer

3

3

CA-2016-138688

2016-06-12

2016-06-16

Second Class

DV-13045

Darrin Van Huff

Corporate

4

4

US-2015-108066

2015-10-11

2015-10-19

Standard Class

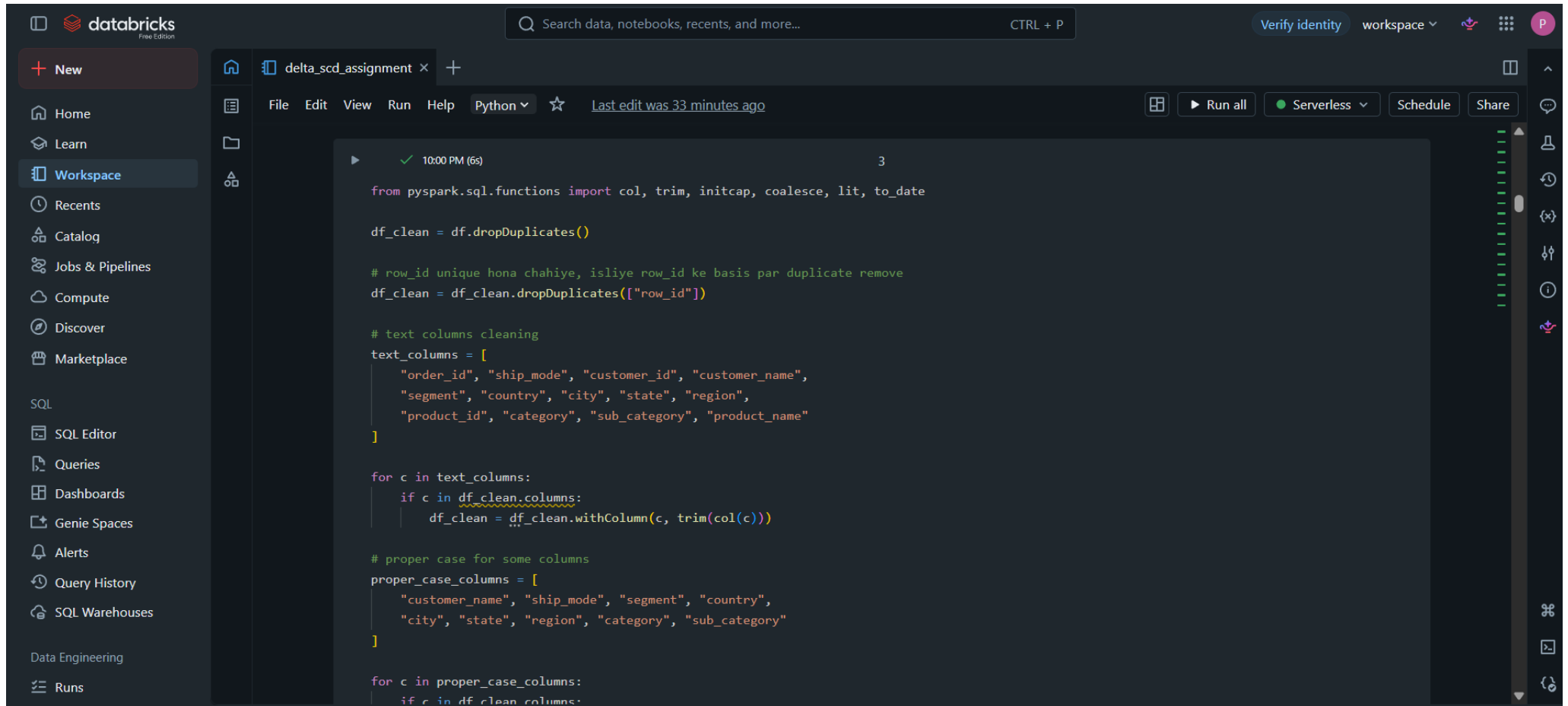
SO-20235

Sean O'Donnell

Consumer

Figure 2: Column name standardization for raw data

Step 2: Basic cleaning



The screenshot shows the Databricks workspace interface. On the left is a sidebar with navigation options: Home, Learn, Workspace (selected), Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Spaces, Alerts, Query History, SQL Warehouses, Data Engineering, and Runs. The main area displays a notebook titled 'delta_scd_assignment'. The code in the notebook is as follows:

```
from pyspark.sql.functions import col, trim, initcap, coalesce, lit, to_date

df_clean = df.dropDuplicates()

# row_id unique hona chahiye, isliye row_id ke basis par duplicate remove
df_clean = df_clean.dropDuplicates(["row_id"])

# text columns cleaning
text_columns = [
    "order_id", "ship_mode", "customer_id", "customer_name",
    "segment", "country", "city", "state", "region",
    "product_id", "category", "sub_category", "product_name"
]

for c in text_columns:
    if c in df_clean.columns:
        df_clean = df_clean.withColumn(c, trim(col(c)))

# proper case for some columns
proper_case_columns = [
    "customer_name", "ship_mode", "segment", "country",
    "city", "state", "region", "category", "sub_category"
]

for c in proper_case_columns:
    if c in df_clean.columns:
```

Figure 3.1: Data cleaning code - remove duplicates, trim text columns and handle data types

Step 2: Basic cleaning

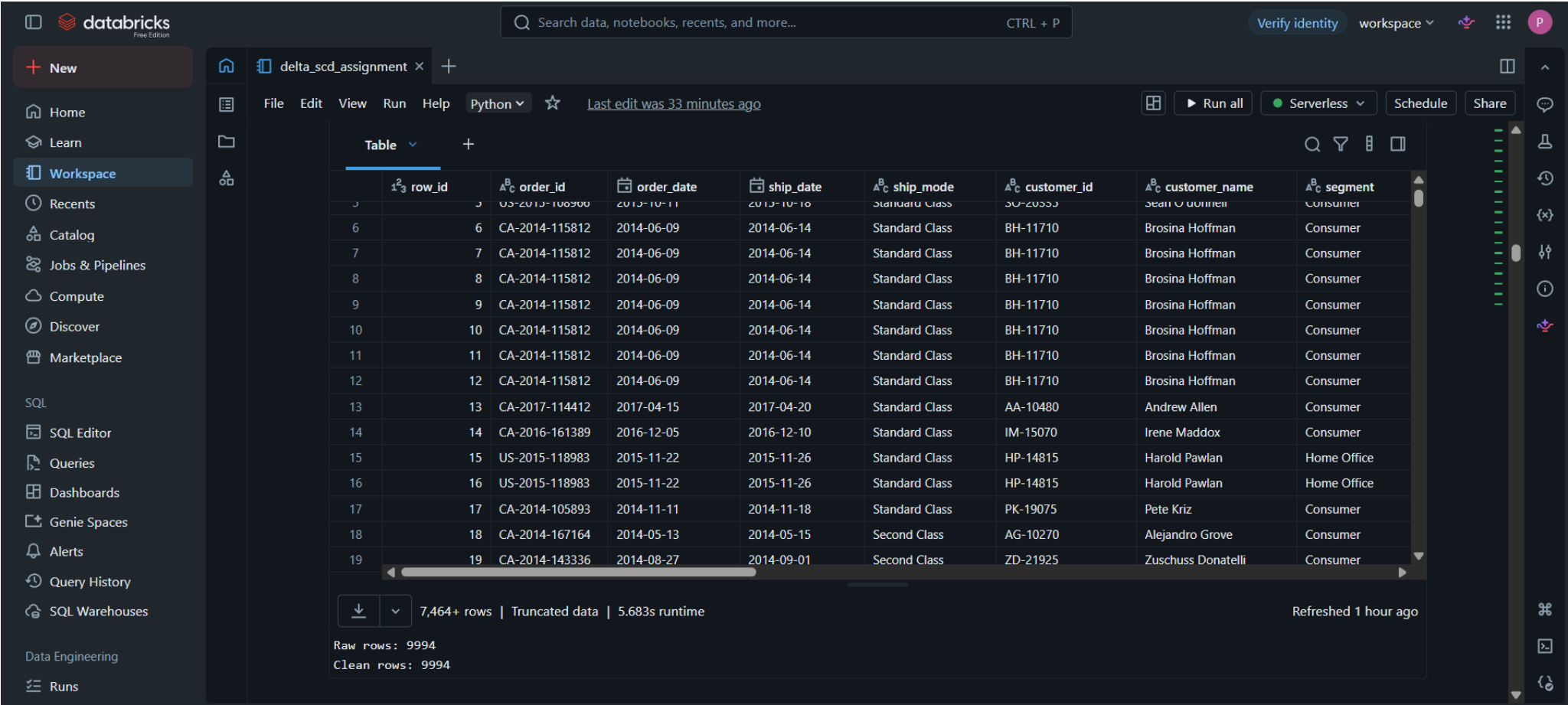
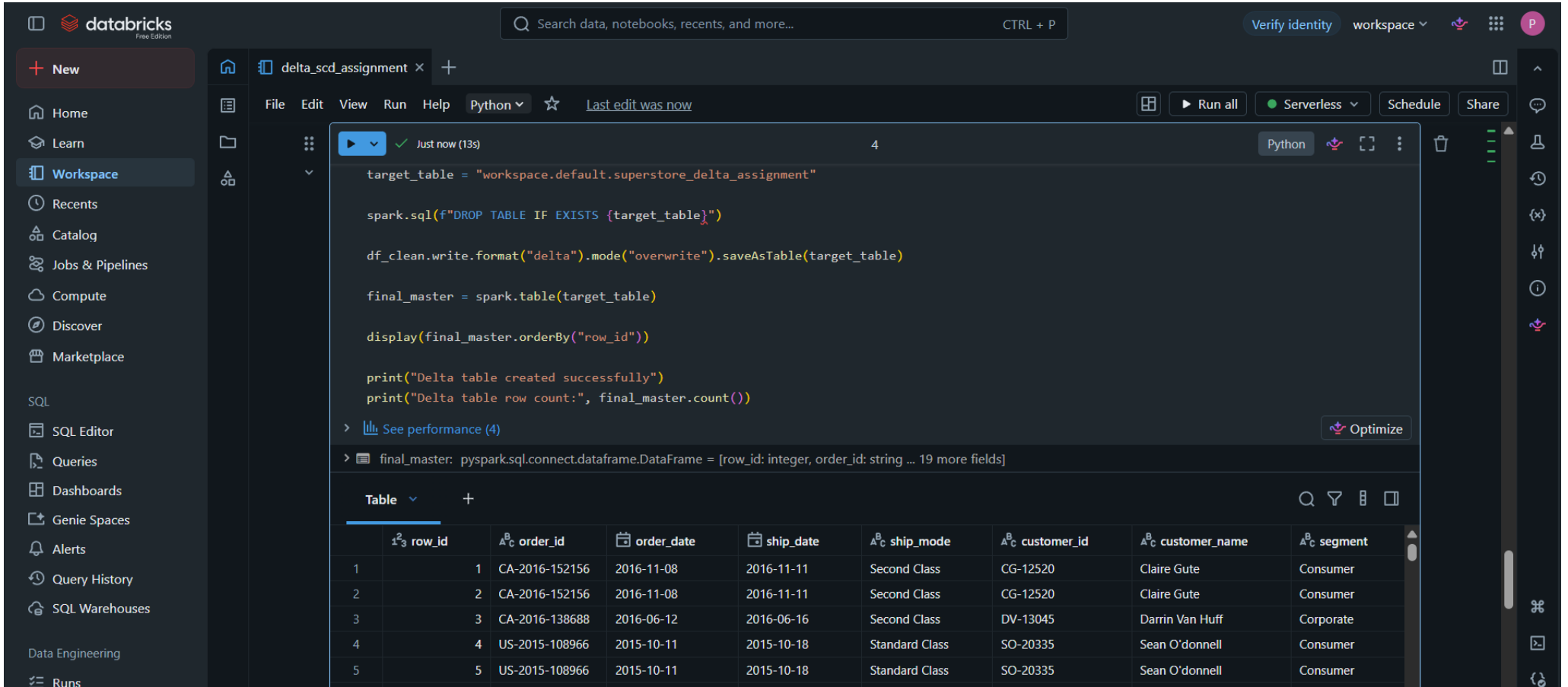


Figure 3.2: Cleaned data output and row-count validation

Step 1.2: Create cleaned Delta table



The screenshot displays the Databricks interface with a Python notebook titled "delta_scd_assignment". The notebook code is as follows:

```
target_table = "workspace.default.superstore_delta_assignment"

spark.sql(f"DROP TABLE IF EXISTS {target_table}")

df_clean.write.format("delta").mode("overwrite").saveAsTable(target_table)

final_master = spark.table(target_table)

display(final_master.orderBy("row_id"))

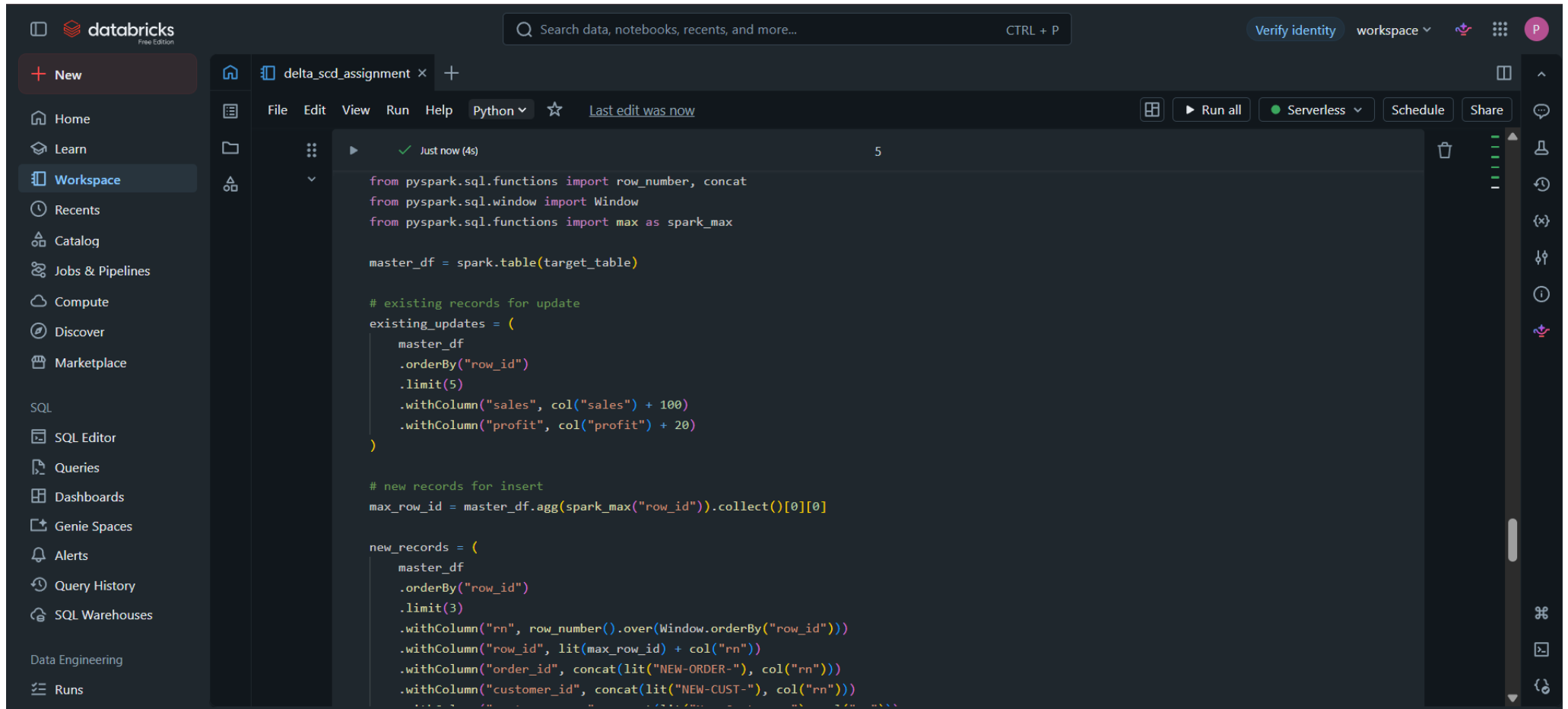
print("Delta table created successfully")
print("Delta table row count:", final_master.count())
```

The output shows the table schema and a preview of 5 rows:

	row_id	order_id	order_date	ship_date	ship_mode	customer_id	customer_name	segment
1	1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer
2	2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer
3	3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate
4	4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'donnell	Consumer
5	5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'donnell	Consumer

Figure 4: Cleaned DataFrame saved as a Delta table

Step 3: Create incremental data



The screenshot shows the Databricks workspace interface. On the left is a sidebar with navigation options: Home, Learn, Workspace (selected), Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Spaces, Alerts, Query History, SQL Warehouses, Data Engineering, and Runs. The main area displays a notebook titled 'delta_scd_assignment'. The notebook has a menu bar with File, Edit, View, Run, and Help. Below the menu bar, there are tabs for 'Python' and 'Last edit was now'. The notebook content is as follows:

```
from pyspark.sql.functions import row_number, concat
from pyspark.sql.window import Window
from pyspark.sql.functions import max as spark_max

master_df = spark.table(target_table)

# existing records for update
existing_updates = (
    master_df
    .orderBy("row_id")
    .limit(5)
    .withColumn("sales", col("sales") + 100)
    .withColumn("profit", col("profit") + 20)
)

# new records for insert
max_row_id = master_df.agg(spark_max("row_id")).collect()[0][0]

new_records = (
    master_df
    .orderBy("row_id")
    .limit(3)
    .withColumn("rn", row_number().over(Window.orderBy("row_id")))
    .withColumn("row_id", lit(max_row_id) + col("rn"))
    .withColumn("order_id", concat(lit("NEW-ORDER-"), col("rn")))
    .withColumn("customer_id", concat(lit("NEW-CUST-"), col("rn")))
)
```

Figure 5.1: Incremental dataset preparation - existing updates and new records

Step 3: Create incremental data

The screenshot shows the Databricks workspace interface. The notebook 'delta_scd_assignment' is open, displaying the following Python code:

```
incremental_df = existing_updates.unionByName(new_records)

display(incremental_df)

print("Incremental records:", incremental_df.count())
```

The output of the code execution is a table with 6 rows and 9 columns. The columns are: row_id, order_id, order_date, ship_date, ship_class, customer_id, customer_name, and segment. The data is as follows:

row_id	order_id	order_date	ship_date	ship_class	customer_id	customer_name	segment
1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer
2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer
3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate
4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'donnell	Consumer
5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'donnell	Consumer
6	NEW-ORDER-1	2016-11-08	2016-11-11	Second Class	NEW-CUST-1	New Customer 1	Consumer

A warning message is also visible in the output area:

```
/databricks/python/lib/python3.12/site-packages/pyspark/sql/connect/expressions.py:1160: UserWarning: WARN WindowExpression: No Partition Defined for Window operation! Moving all data to a single partition, this can cause serious performance degradation.
warnings.warn()
```

Figure 5.2: Incremental dataset output with updated and new records

Step 4: Apply MERGE operation

The screenshot displays the Databricks workspace interface. On the left, a sidebar contains navigation links: Home, Learn, Workspace (selected), Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Spaces, Alerts, Query History, SQL Warehouses, Data Engineering, and Runs. The main workspace area shows a notebook titled 'delta_scd_assignment'. The notebook's menu bar includes File, Edit, View, Run, Help, and a Python language selector. A search bar at the top right contains the text 'Search data, notebooks, recents, and more...' and a 'CTRL + P' shortcut. The notebook content is a Python script that performs a MERGE operation on a Delta Lake table. The script uses the following SQL code:

```
spark.sql(f"""
MERGE INTO {target_table} AS target
USING superstore_incremental_view AS source
ON target.row_id = source.row_id

WHEN MATCHED THEN
  UPDATE SET *

WHEN NOT MATCHED THEN
  INSERT *
""")

after_merge_count = spark.table(target_table).count()

print("MERGE operation completed successfully")
print("Before merge row count:", before_merge_count)
print("After merge row count:", after_merge_count)
```

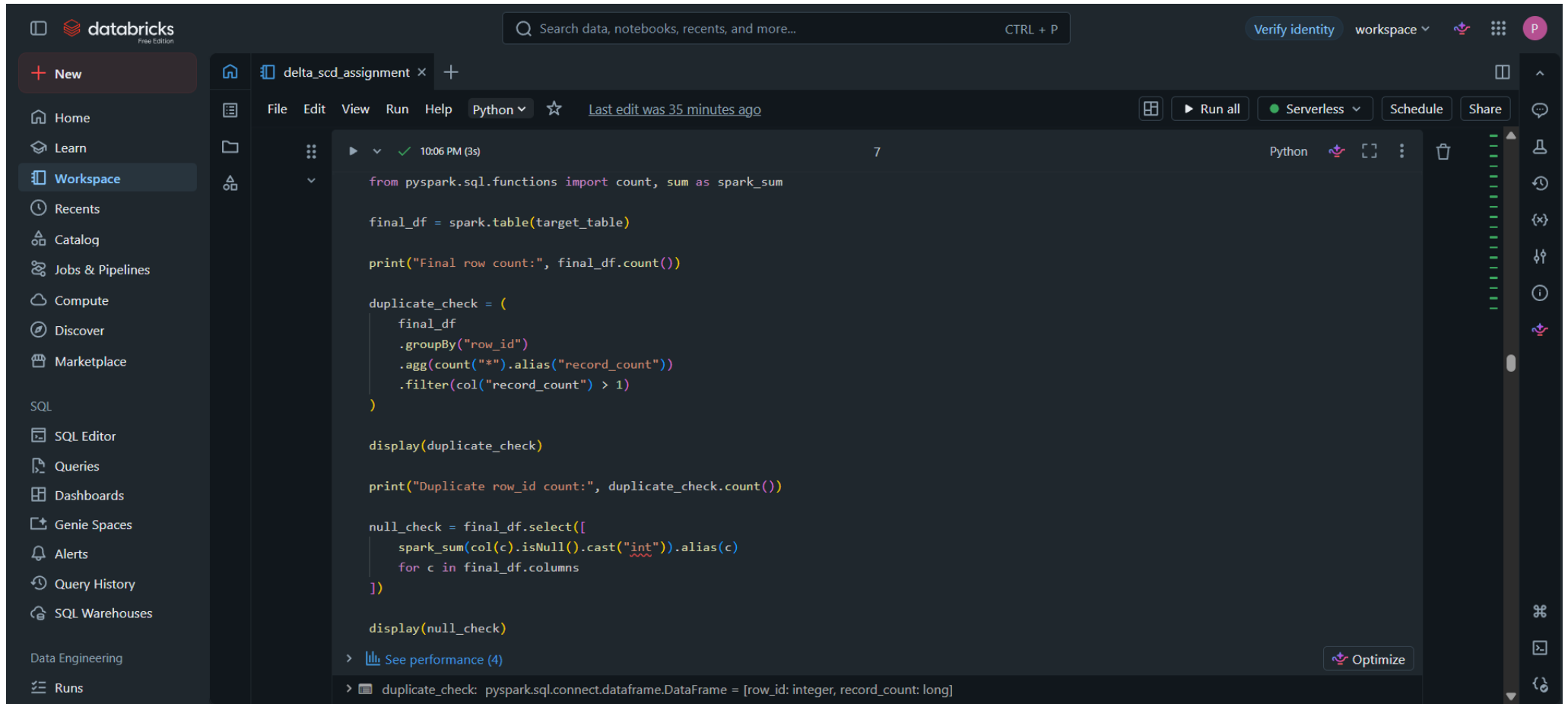
The script also includes a performance optimization link: [See performance \(4\)](#). The output of the script is displayed at the bottom of the notebook, showing a warning message and the results of the MERGE operation:

```
/databricks/python/lib/python3.12/site-packages/pyspark/sql/connect/expressions.py:1160: UserWarning: WARN WindowExpression: No Partition Defined for Window operation! Moving all data to a single partition, this can cause serious performance degradation.
  warnings.warn(
MERGE operation completed successfully
Before merge row count: 9994
After merge row count: 9997
```

The right sidebar of the workspace contains various utility icons, including a search icon, a refresh icon, a share icon, and a help icon.

Figure 6: Delta Lake MERGE operation for SCD Type 1 upsert

Step 5: Validate results



The screenshot shows the Databricks workspace interface. On the left is a sidebar with navigation options: Home, Learn, Workspace (selected), Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Spaces, Alerts, Query History, SQL Warehouses, Data Engineering, and Runs. The main area displays a notebook titled 'delta_scd_assignment'. The code in the notebook is as follows:

```
from pyspark.sql.functions import count, sum as spark_sum

final_df = spark.table(target_table)

print("Final row count:", final_df.count())

duplicate_check = (
    final_df
    .groupBy("row_id")
    .agg(count("*").alias("record_count"))
    .filter(col("record_count") > 1)
)

display(duplicate_check)

print("Duplicate row_id count:", duplicate_check.count())

null_check = final_df.select([
    spark_sum(col(c).isNull().cast("int")).alias(c)
    for c in final_df.columns
])

display(null_check)
```

Below the code, there is a link to 'See performance (4)' and a status bar indicating 'duplicate_check: pyspark.sql.connect.dataframe.DataFrame = [row_id: integer, record_count: long]'. The top of the interface includes a search bar, a 'Verify identity' button, and a 'workspace' dropdown menu.

Figure 7.1: Final table validation code - row count, duplicate check and null check

Step 5: Validate results

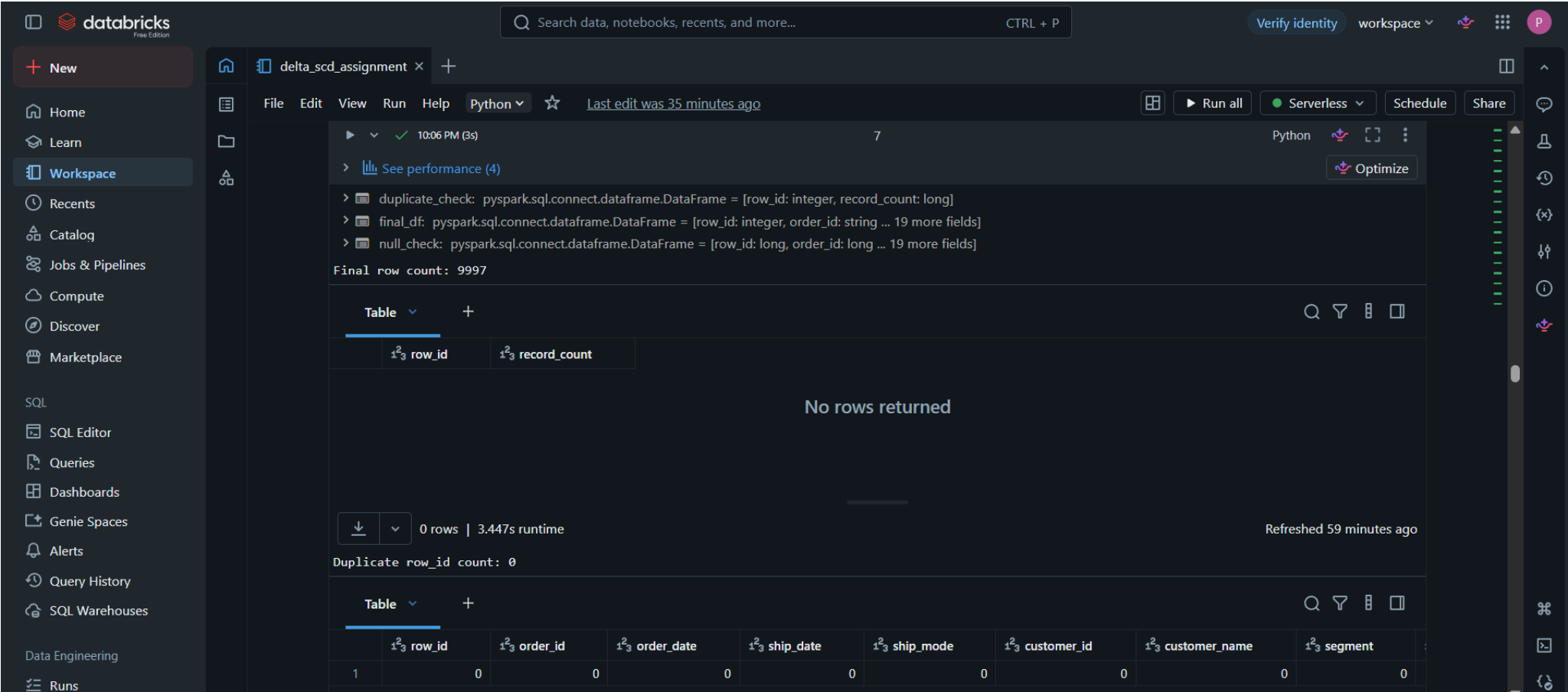


Figure 7.2: Validation output showing final row count and zero duplicate row IDs

Step 6: Display final dataset

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Spaces

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

delta_scd_assignment

File Edit View Run Help Python Last edit was 36 minutes ago

display(final_df.orderBy("row_id"))

See performance (1)

Table

	row_id	order_id	order_date	ship_date	ship_mode	customer_id	customer_name	segment
2	2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer
3	3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate
4	4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'donnell	Consumer
5	5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'donnell	Consumer
6	6	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
7	7	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
8	8	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
9	9	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
10	10	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
11	11	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
12	12	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
13	13	CA-2017-114412	2017-04-15	2017-04-20	Standard Class	AA-10480	Andrew Allen	Consumer
14	14	CA-2016-161389	2016-12-05	2016-12-10	Standard Class	IM-15070	Irene Maddox	Consumer
15	15	US-2015-118983	2015-11-22	2015-11-26	Standard Class	HP-14815	Harold Pawlan	Home Office

Figure 8: Final SCD Type 1 Delta table after MERGE

Step 6: Display summary

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Spaces

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

delta_scd_assignment

File Edit View Run Help Python ☆ Last edit was now

10:08 PM (5s)

9

```
summary_data = [
    ("Raw data rows", df_raw.count()),
    ("Cleaned data rows", df_clean.count()),
    ("Incremental data rows", incremental_df.count()),
    ("Final Delta table rows", final_df.count()),
    ("Duplicate row_id count", duplicate_check.count())
]

summary_df = spark.createDataFrame(summary_data, ["Metric", "Value"])

display(summary_df)
```

See performance (6)Optimize

summary_df: pyspark.sql.connect.dataframe.DataFrame = [Metric: string, Value: long]

/databricks/python/lib/python3.12/site-packages/pyspark/sql/connect/expressions.py:1160: UserWarning: WARN WindowExpression: No Partition Defined for Window operation! Moving all data to a single partition, this can cause serious performance degradation.
warnings.warn()

Table

+

	Metric	Value
1	Raw data rows	9994
2	Cleaned data rows	9994
3	Incremental data rows	8

Figure 9.1: Summary DataFrame creation for assignment metrics

Step 6: Display summary

The screenshot shows the Databricks workspace interface. The notebook code is as follows:

```
10:08 PM (5s) 9
1. And select count rows, final_row_count,
2. ("Duplicate row_id count", duplicate_check.count())
3. ]
4.
5. summary_df = spark.createDataFrame(summary_data, ["Metric", "Value"])
6.
7. display(summary_df)
8.
9. > See performance (6)
10. > summary_df: pyspark.sql.connect.dataframe.DataFrame = [Metric: string, Value: long]
11. /databricks/python/lib/python3.12/site-packages/pyspark/sql/connect/expressions.py:1160: UserWarning: WARN WindowExpression: No Partition Defined for
12. Window operation! Moving all data to a single partition, this can cause serious performance degradation.
13. warnings.warn(
```

The output is a table with 5 rows:

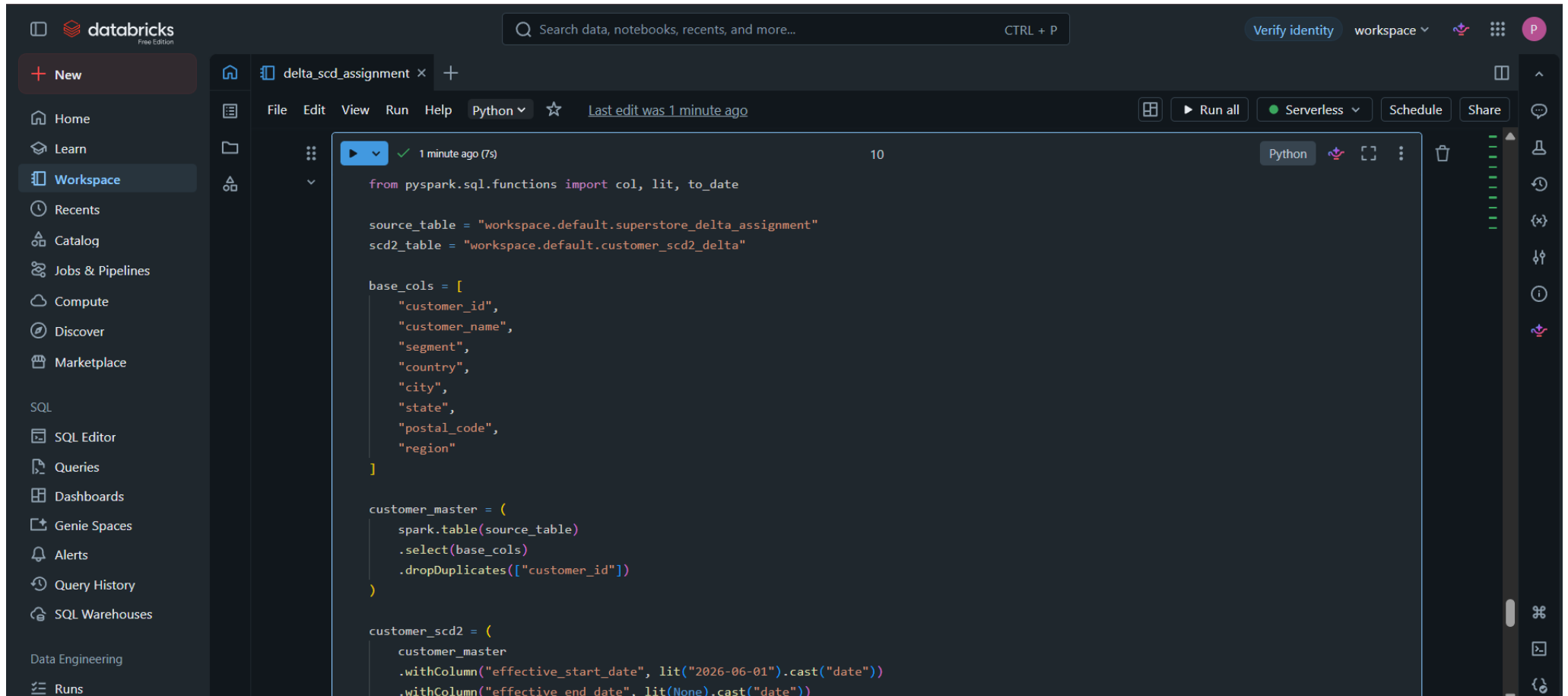
	Metric	Value
1	Raw data rows	9994
2	Cleaned data rows	9994
3	Incremental data rows	8
4	Final Delta table rows	9997
5	Duplicate row_id cou...	0

5 rows | 4.818s runtime

Refreshed 7 minutes ago

Figure 9.2: Summary output for SCD Type 1 processing

Step7: SCD Type 2 implementation



```
from pyspark.sql.functions import col, lit, to_date

source_table = "workspace.default.superstore_delta_assignment"
scd2_table = "workspace.default.customer_scd2_delta"

base_cols = [
    "customer_id",
    "customer_name",
    "segment",
    "country",
    "city",
    "state",
    "postal_code",
    "region"
]

customer_master = (
    spark.table(source_table)
    .select(base_cols)
    .dropDuplicates(["customer_id"])
)

customer_scd2 = (
    customer_master
    .withColumn("effective_start_date", lit("2026-06-01").cast("date"))
    .withColumn("effective_end_date", lit(None).cast("date"))
)
```

Figure 10.1: SCD Type 2 customer table creation from master data

SCD Type 2 implementation

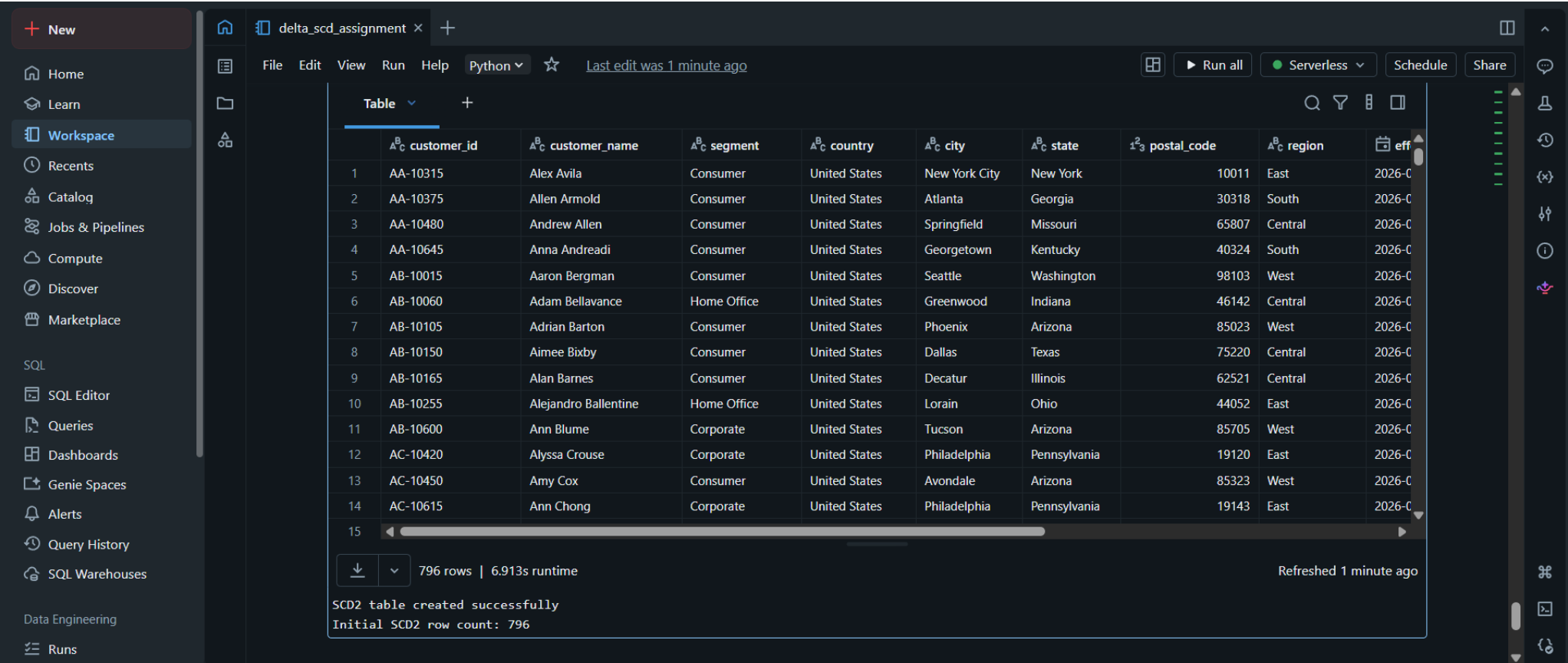
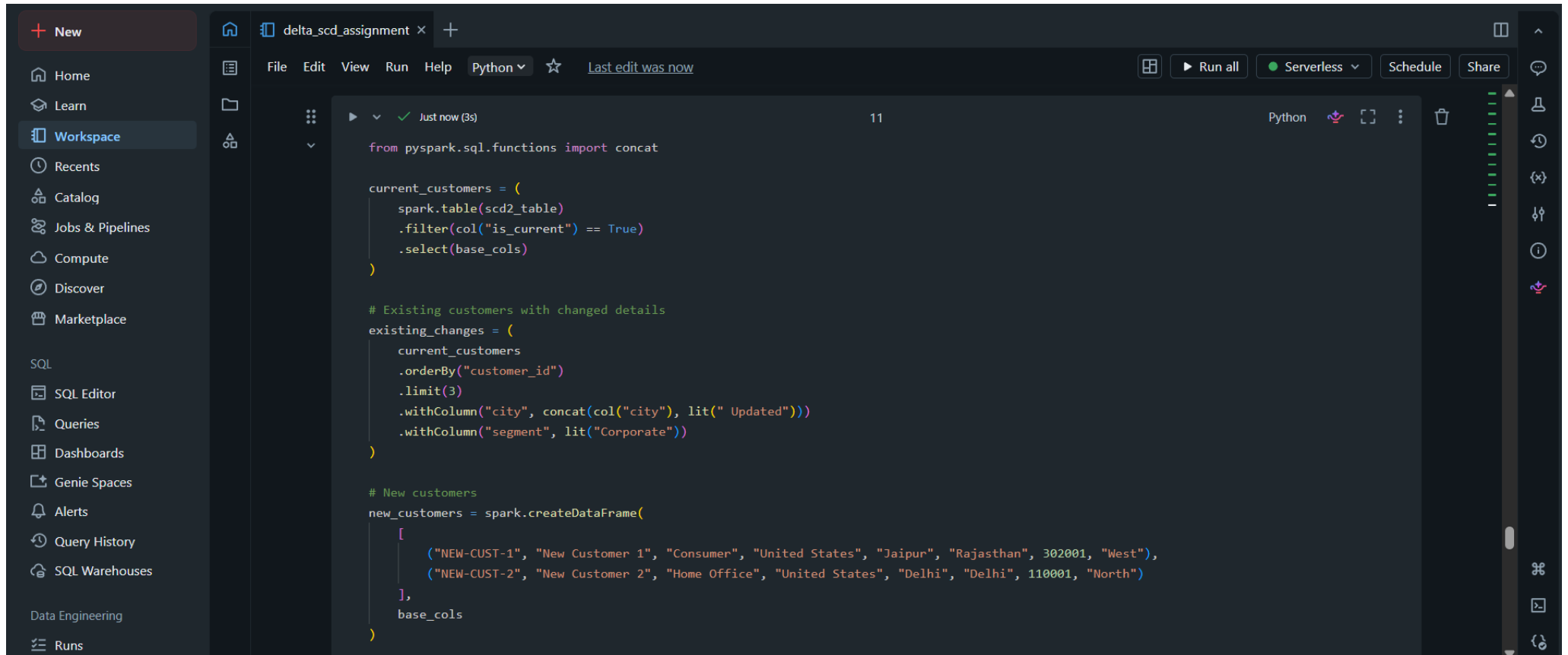


Figure 10.2: Initial SCD Type 2 table output and row count

SCD Type 2 implementation



The screenshot displays a Databricks workspace interface. On the left is a sidebar with navigation options: Home, Learn, Workspace (selected), Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Spaces, Alerts, Query History, SQL Warehouses, Data Engineering, and Runs. The main area shows a file named 'delta_scd_assignment' with a Python editor. The code implements SCD Type 2 logic by selecting current customers, identifying changes, and creating new customer records.

```
from pyspark.sql.functions import concat

current_customers = (
    spark.table(sc2_table)
    .filter(col("is_current") == True)
    .select(base_cols)
)

# Existing customers with changed details
existing_changes = (
    current_customers
    .orderBy("customer_id")
    .limit(3)
    .withColumn("city", concat(col("city"), lit(" Updated")))
    .withColumn("segment", lit("Corporate"))
)

# New customers
new_customers = spark.createDataFrame(
    [
        ("NEW-CUST-1", "New Customer 1", "Consumer", "United States", "Jaipur", "Rajasthan", 302001, "West"),
        ("NEW-CUST-2", "New Customer 2", "Home Office", "United States", "Delhi", "Delhi", 110001, "North")
    ],
    base_cols
)
```

Figure 11.1: SCD Type 2 incremental data preparation

SCD Type 2 implementation

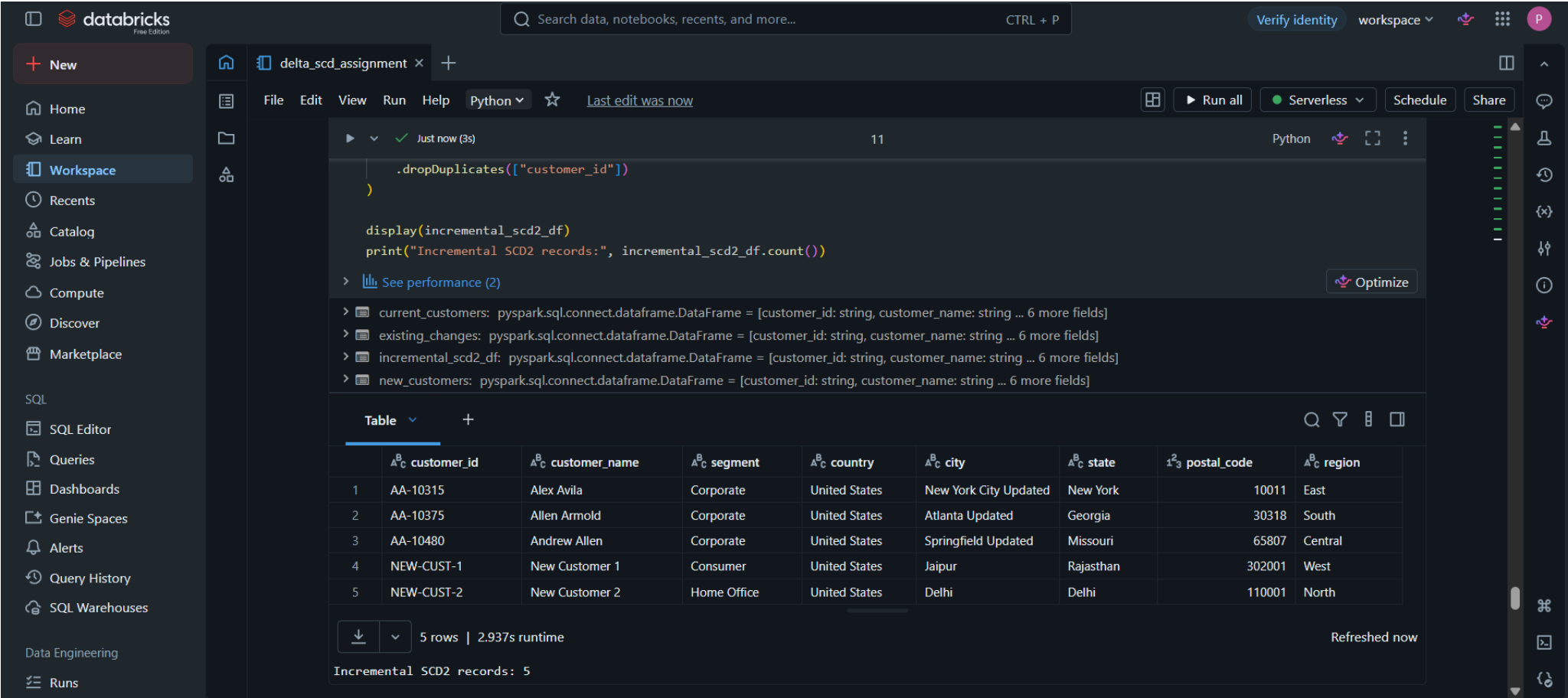
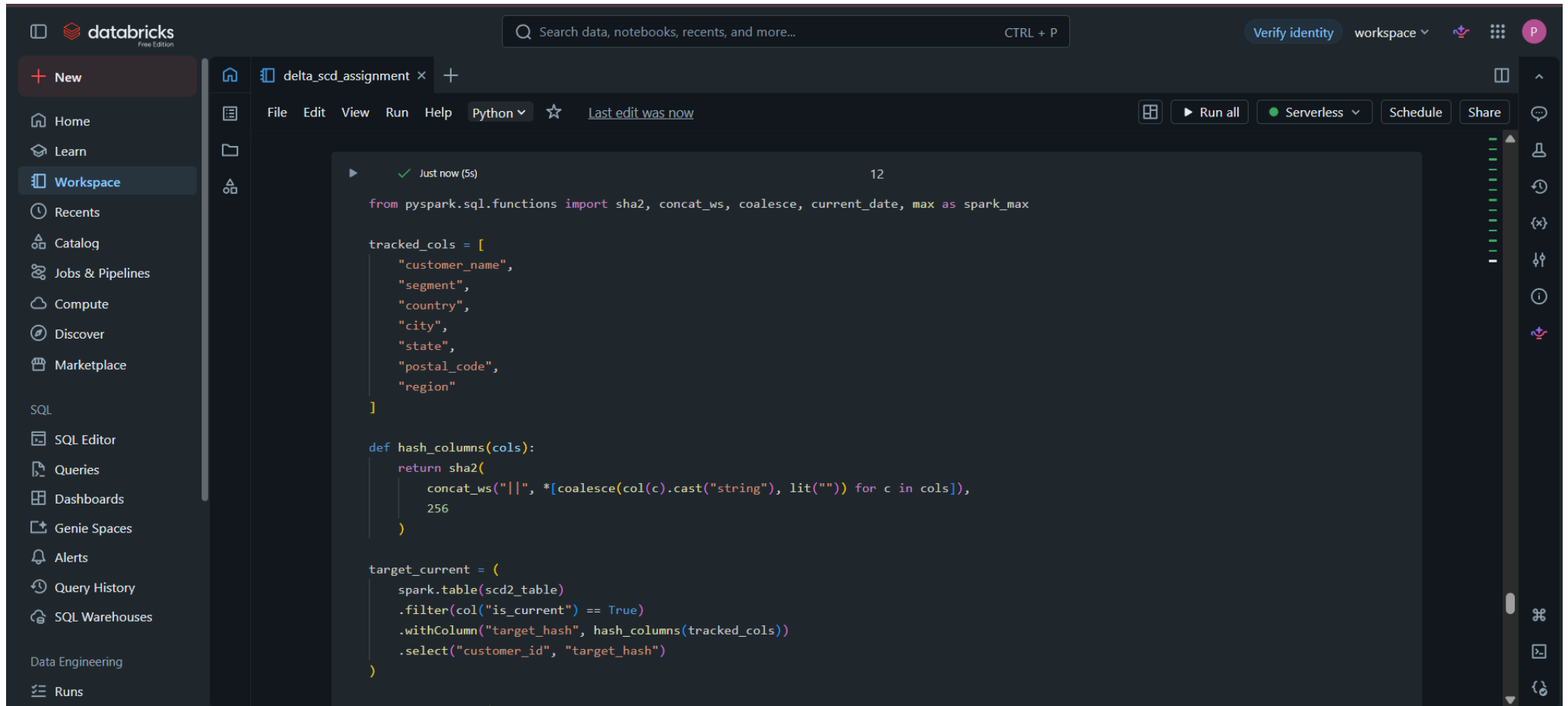


Figure 11.2: SCD Type 2 incremental records displayed

SCD Type 2 implementation



The screenshot displays the Databricks workspace interface. On the left, a sidebar contains navigation links: Home, Learn, Workspace (selected), Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Spaces, Alerts, Query History, SQL Warehouses, Data Engineering, and Runs. The top navigation bar includes the Databricks logo, a search bar, and buttons for 'Verify identity', 'workspace', and a user profile icon. The main area shows a notebook titled 'delta_scd_assignment'. The notebook's toolbar includes 'File', 'Edit', 'View', 'Run', 'Help', a language selector set to 'Python', a star icon, and a link to 'Last edit was now'. The notebook content consists of a Python script for SCD Type 2 implementation. The script imports functions from pyspark.sql.functions, defines a list of tracked columns, creates a hash_columns function, and constructs a target_current query. The script is executed, showing a successful status 'Just now (5s)' and a line count of 12.

```
from pyspark.sql.functions import sha2, concat_ws, coalesce, current_date, max as spark_max

tracked_cols = [
    "customer_name",
    "segment",
    "country",
    "city",
    "state",
    "postal_code",
    "region"
]

def hash_columns(cols):
    return sha2(
        concat_ws("||", *[coalesce(col(c).cast("string"), lit("")) for c in cols]),
        256
    )

target_current = (
    spark.table(scd2_table)
    .filter(col("is_current") == True)
    .withColumn("target_hash", hash_columns(tracked_cols))
    .select("customer_id", "target_hash")
)
```

Figure 12.1: Hash-based change detection logic for SCD Type 2

SCD Type 2 implementation

The screenshot shows the Databricks workspace interface. On the left is a sidebar with navigation options: Home, Learn, Workspace (selected), Recents, Catalog, Jobs & Pipelines, Compute, Discover, and Marketplace. Below these are SQL-related options: SQL Editor, Queries, Dashboards, Genie Spaces, Alerts, Query History, and SQL Warehouses. At the bottom are Data Engineering and Runs. The main area displays a notebook titled 'delta_scd_assignment'. The notebook has a menu bar with File, Edit, View, Run, and Help. The code cell shows a Python script that has just executed, displaying 'Just now (5s)' and '12' records. The code includes a `display(records_to_insert)` and a `print("Records to insert in SCD2:", records_to_insert.count())`. Below the code, a table view shows 5 rows of data selected for insertion. The table has columns: customer_id, customer_name, segment, country, city, state, postal_code, and region. The records are: 1. AA-10315, Alex Avila, Corporate, United States, New York City Updated, New York, 10011, East; 2. AA-10375, Allen Arnold, Corporate, United States, Atlanta Updated, Georgia, 30318, South; 3. AA-10480, Andrew Allen, Corporate, United States, Springfield Updated, Missouri, 65807, Central; 4. NEW-CUST-1, New Customer 1, Consumer, United States, Jaipur, Rajasthan, 302001, West; 5. NEW-CUST-2, New Customer 2, Consumer, United States, Delhi, Delhi, 110001, North. The table view includes a search bar, filters, and a refresh button. The status bar at the bottom indicates '5 rows | 4.570s runtime' and 'Refreshed now'.

	customer_id	customer_name	segment	country	city	state	postal_code	region
1	AA-10315	Alex Avila	Corporate	United States	New York City Updated	New York	10011	East
2	AA-10375	Allen Arnold	Corporate	United States	Atlanta Updated	Georgia	30318	South
3	AA-10480	Andrew Allen	Corporate	United States	Springfield Updated	Missouri	65807	Central
4	NEW-CUST-1	New Customer 1	Consumer	United States	Jaipur	Rajasthan	302001	West
5	NEW-CUST-2	New Customer 2	Consumer	United States	Delhi	Delhi	110001	North

Figure 12.2: Records selected for insertion in SCD Type 2

SCD Type 2 implementation

The screenshot displays the Databricks workspace environment. On the left sidebar, navigation options include Home, Learn, Workspace (selected), Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL Editor, Queries, Dashboards, Genie Spaces, Alerts, Query History, SQL Warehouses, Data Engineering, and Runs.

The main area shows a notebook titled "delta_scd_assignment". The top toolbar includes File, Edit, View, Run, Help, Python (selected), Last edit was now, Run all, Serverless (selected), Schedule, and Share. Below the menu bar, a table preview is visible:

	customer_id	Home Office	United States	Delhi	Delhi	110001	North	2026-07-06
5	omer 2							

Below the table, it indicates "5 rows | 4.570s runtime" and "Records to insert in SCD2: 5". A status bar at the bottom right says "Refreshed 1 minute ago".

The notebook code is as follows:

```
records_to_insert.createOrReplaceTempView("scd2_changes_view")

spark.sql(f"""
MERGE INTO {scd2_table} AS target
USING scd2_changes_view AS source
ON target.customer_id = source.customer_id
AND target.is_current = true

WHEN MATCHED THEN
    UPDATE SET
        target.is_current = false,
        target.effective_end_date = date_sub(source.effective_start_date, 1)
""")

print("Old current records expired successfully")
```

A performance link "> See performance (2)" and an "Optimize" button are located below the code. The execution result at the bottom states "Old current records expired successfully".

Figure 13: SCD Type 2 MERGE operation to expire old current records

SCD Type 2 implementation



The screenshot shows a Databricks notebook interface with a dark theme. At the top, there's a toolbar with a play button, a green checkmark, the text 'Just now (4s)', the number '14', a 'Python' language selector, and icons for undo, redo, and a trash can. The main area contains a code cell with the following Python code:

```
records_to_insert_typed = records_to_insert.withColumn("postal_code", col("postal_code").cast("int"))

records_to_insert_typed.write.format("delta").mode("append").saveAsTable(scd2_table)

print("New SCD2 version records inserted successfully")
```

Below the code, there are two expandable sections. The first is labeled '> See performance (1)' and the second is labeled '> records_to_insert_typed: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, customer_name: string ... 10 more fields]'. The output of the code cell is 'New SCD2 version records inserted successfully'.

Figure 14: New SCD Type 2 version records inserted successfully

Additional SCD Type 2 implementation

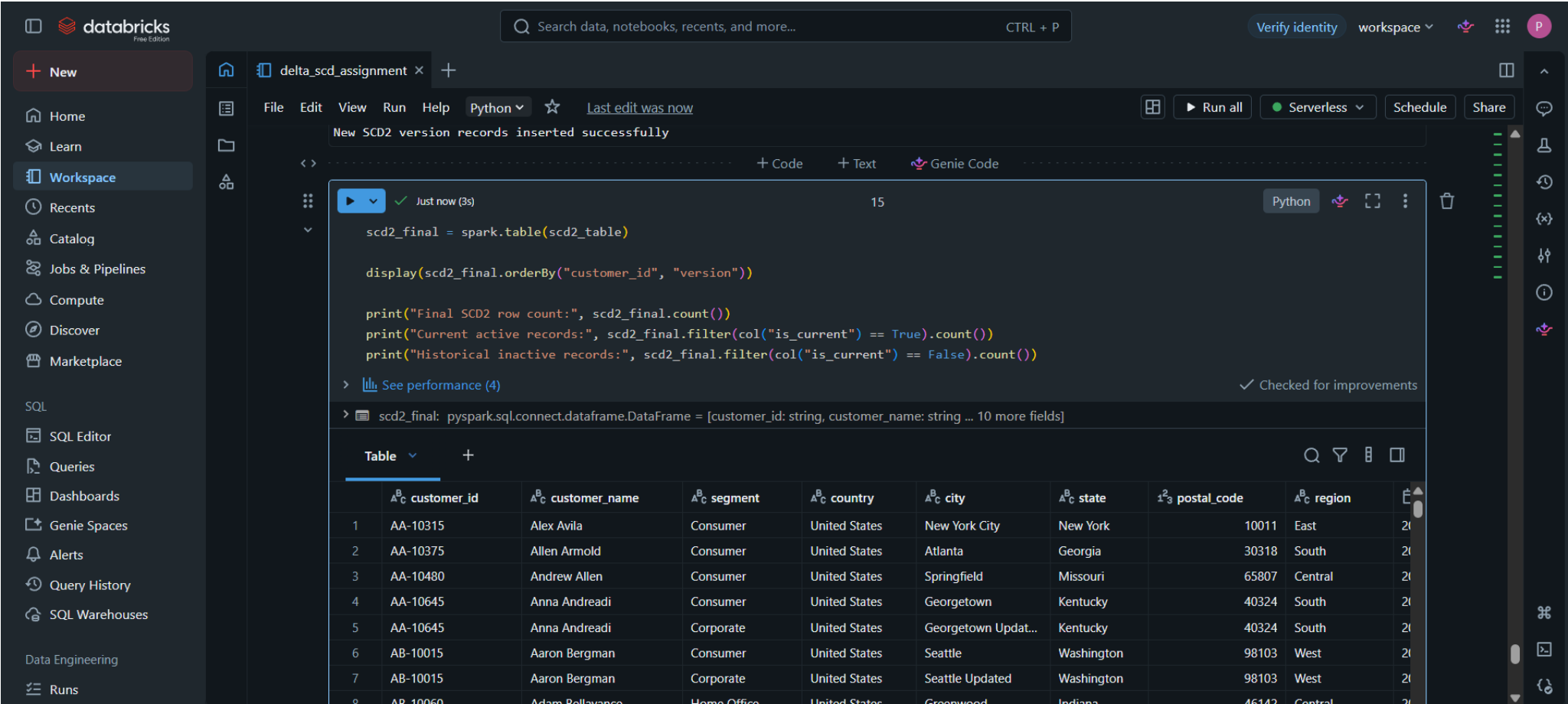


Figure 15.1: Final SCD Type 2 table display after updates and inserts

Additional SCD Type 2 implementation

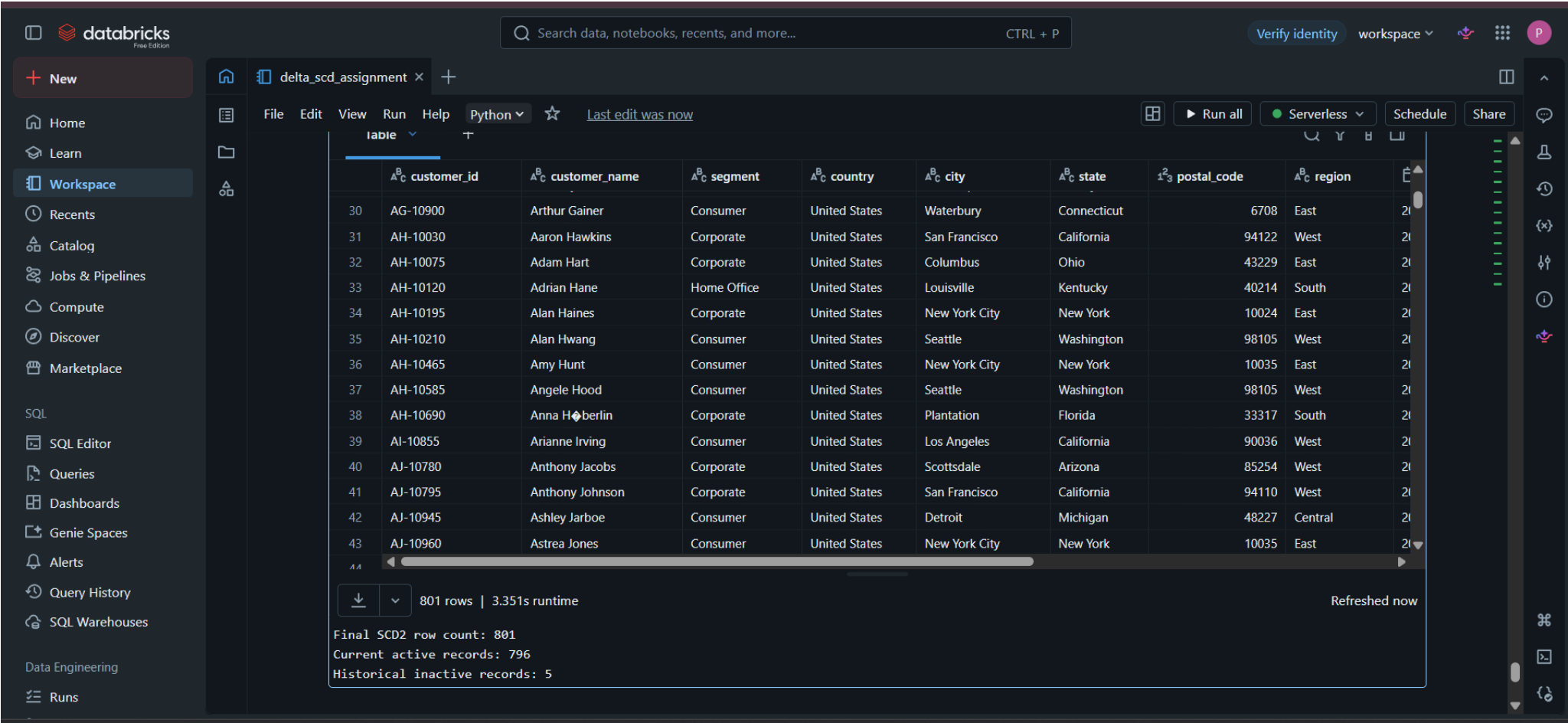


Figure 15.2: SCD Type 2 final row count and active/inactive record summary

Additional SCD Type 2 validation

Just now (2s)

16

Python

```
from pyspark.sql.functions import count

current_duplicate_check = (
    scd2_final
    .filter(col("is_current") == True)
    .groupBy("customer_id")
    .agg(count("*").alias("current_record_count"))
    .filter(col("current_record_count") > 1)
)

display(current_duplicate_check)

print("Customers having more than one current record:", current_duplicate_check.count())
```

> [See performance \(2\)](#)

Optimize

>

current_duplicate_check: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, current_record_count: long]

Table +

	A^B_C customer_id	1²₃ current_record_count
1	AA-10645	2
2	AB-10015	2
3	AB-10060	2

Figure 16.1: Current-record duplicate validation for SCD Type 2

Additional SCD Type 2 validation

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Spaces

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

delta_scd_assignment

File Edit View Run Help Python Last edit was now

Run all

Serverless

Schedule

Share

Just now (2s)

16

Python

```
df.filter(col("is_current") == True)
  .groupBy("customer_id")
  .agg(count("*").alias("current_record_count"))
  .filter(col("current_record_count") > 1)

display(current_duplicate_check)

print("Customers having more than one current record:", current_duplicate_check.count())
```

See performance (2)

Optimize

current_duplicate_check: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, current_record_count: long]

Table

	customer_id	current_record_count
1	AA-10645	2
2	AB-10015	2
3	AB-10060	2

Download

3 rows | 1.731s runtime

Refreshed now

Customers having more than one current record: 3

Figure 16.2: SCD Type 2 duplicate check output showing customers with more than one current record

Additional SCD Type 2 validation

databricks

Free Edition

+

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Spaces

Alerts

Query History

SQL Warehouses

delta_scd_assignment

File Edit View Run Help Python ☆ Last edit was now

display(
 scd2_final
 .filter(col("version") > 1)
 .orderBy("customer_id", "version")
)

See performance (1)

Checked for improvements

Table

	customer_id	customer_name	segment	country	city	state	postal_code	region	
1	AA-10645	Anna Andreadi	Corporate	United States	Georgetown Updat...	Kentucky	40324	South	20
2	AB-10015	Aaron Bergman	Corporate	United States	Seattle Updated	Washington	98103	West	20
3	AB-10060	Adam Bellavance	Corporate	United States	Greenwood Updated	Indiana	46142	Central	20
4	NEW-CUST-1	New Customer 1	Consumer	United States	Jaipur	Rajasthan	302001	West	20
5	NEW-CUST-2	New Customer 2	Consumer	United States	Jaipur	Rajasthan	302001	West	20

5 rows | 0.957s runtime

Refreshed now

Figure 17: Historical SCD Type 2 records with version greater than 1

Final assignment summary

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Spaces

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

delta_scd_assignment

File Edit View Run Help Python Last edit was now

Just now (7s)

18

Python

```
summary_data = [
    ("Original raw data rows", df_raw.count()),
    ("Cleaned Delta table rows before merge", df_clean.count()),
    ("Incremental records used for SCD1", incremental_df.count()),
    ("Final SCD1 table rows", final_df.count()),
    ("Duplicate row_id after SCD1", duplicate_check.count()),
    ("Final SCD2 table rows", scd2_final.count()),
    ("Current active SCD2 records", scd2_final.filter(col("is_current") == True).count()),
    ("Historical inactive SCD2 records", scd2_final.filter(col("is_current") == False).count()),
    ("Customers having more than one current SCD2 record", current_duplicate_check.count())
]

assignment_summary = spark.createDataFrame(summary_data, ["Metric", "Value"])

display(assignment_summary)
```

See performance (10)Optimize

assignment_summary: pyspark.sql.connect.dataframe.DataFrame = [Metric: string, Value: long]

/databricks/python/lib/python3.12/site-packages/pyspark/sql/connect/expressions.py:1160: UserWarning: WARN WindowExpression: No Partition Defined for Window operation! Moving all data to a single partition, this can cause serious performance degradation.

warnings.warn(

Table

Metric

Value

Figure 18.1: Final assignment summary DataFrame creation

Final assignment summary

The screenshot shows a Databricks notebook interface with a dark theme. The notebook is named "delta_scd_assignment". The top toolbar includes buttons for "File", "Edit", "View", "Run", "Help", a "Python" language selector, a star icon, and a link "Last edit was now". On the right, there are buttons for "Run all", "Serverless", "Schedule", and "Share".

The main area displays the execution of a Python cell. It starts with a play button icon, a checkmark, and the text "Just now (7s)". The code executed is `display(assignment_summary)`. Below the code, there is a link "See performance (10)" and an "Optimize" button. A message indicates the data type: `assignment_summary: pyspark.sql.connect.dataframe.DataFrame = [Metric: string, Value: long]`. A warning message is shown: `/databricks/python/lib/python3.12/site-packages/pyspark/sql/connect/expressions.py:1160: UserWarning: WARN WindowExpression: No Partition Defined for Window operation! Moving all data to a single partition, this can cause serious performance degradation.` followed by `warnings.warn()`.

Below the code, a table view is displayed. It has a "Table" dropdown and a "+" icon. The table has two columns: "Metric" and "Value". It contains 9 rows of data. At the bottom of the table view, it says "9 rows | 7.045s runtime". A "Refreshed now" button is located at the bottom right of the table view.

	Metric	Value
1	Original raw data rows	9994
2	Cleaned Delta table rows before merge	9994
3	Incremental records used for SCD1	8
4	Final SCD1 table rows	9997
5	Duplicate row_id after SCD1	0
6	Final SCD2 table rows	801
7	Current active SCD2 records	796
8	Historical inactive SCD2 records	5
9	Customers having more than one current SCD2 record	3

Figure 18.2: Final assignment summary output